

1 Abstract

This report details a rigorous data integrity audit performed on a high-resolution HPLC chromatography dataset intended for machine learning applications. The primary objective was to evaluate whether the dataset meets strict biopharmaceutical standards regarding signal quality, completeness, and unit consistency. Analysis revealed severe data integrity failures, including a 65.6% missingness rate in critical variables such as ‘Conc_B_ml’ and ‘pH_ml’, and the presence of negative value artifacts in UV absorbance columns, indicating data entry errors or baseline noise. Furthermore, a significant ‘Modeling Feasibility Gap’ was identified, where only 24.32% of rows contain both ‘chromatography_stage’ and ‘Sample_Code’, rendering sample-specific modeling infeasible for the majority of the data. The methodology involved a three-step process: a completeness audit, dual-validation artifact detection, and the derivation of a high-completeness feature set. Based on these findings, the ‘Fit for Use’ dataset was constructed by excluding variables with missingness exceeding 95% and retaining only those with valid, non-negative units. Consequently, the final usable dataset comprises 1,084,069 rows utilizing a reduced variable set of ‘run_no’, ‘UV_1_280_mAU’, ‘UV_2_260_mAU’, ‘Cond_mS_cm’, ‘Conc_B_%’, and ‘pH_pH’. This report concludes that while a high-quality feature set can be derived, the dataset cannot currently support granular sample tracking without aggressive data reconstruction.

2 Introduction

In the context of biopharmaceutical manufacturing, the reliability of chromatographic data is paramount for downstream machine learning and process optimization. However, datasets derived from high-resolution fraction-level time-series often suffer from critical integrity issues that compromise model reliability. This analysis focuses on a dataset containing variables such as concentration, pH, UV absorbance, and process metadata, which exhibits severe completeness gaps and unit validity failures. Specifically, key variables like ‘Conc_B_ml’ and ‘pH_ml’ demonstrate missingness rates exceeding 65%, while unit confusion has led to negative absorbance values in columns labeled with ‘_ml’, suggesting mislabeling between volume and absorbance metrics. The motivation for this analysis was to quantify these failures and determine if the dataset is ‘fit for use’ by strictly adhering to biopharmaceutical standards. The goal was to identify specific data integrity failures—such as missingness exceeding 65% in key variables, unit mislabeling, and missing process metadata—that would render predictive modeling unreliable. The analysis aims to provide a quantified decision on modeling feasibility and to guide the exclusion of corrupted variables rather than introducing bias through aggressive imputation, ensuring that only variables with ≥95% completeness and valid units are utilized for feature engineering.

3 Methodology

The data analysis workflow was structured into three distinct steps to ensure a comprehensive evaluation of the dataset’s integrity. Step 1 involved a Data Integrity and Completeness Audit. This phase calculated exact row percentages for missingness and unit validity across all variables, including ‘run_no’, ‘chromatography_stage’, ‘Sample_Code’, and various concentration and pH metrics. The objective was to measure the extent of failure without fixing the data, explicitly defining the ‘fit for use’ dataset boundaries. Step 2 focused on Dual-Validation Artifact Detection. This step analyzed signal integrity for validated variables, specifically ‘UV_1_280_mAU’ and ‘UV_2_260_mAU’, to detect negative value artifacts. Additionally, a correlation analysis was performed between ‘UV_1_280_mAU’ and ‘volume_ml’ to verify potential unit mislabeling, validating the plan’s decision to filter rather than impute. Step 3 involved the High-Completeness Feature Set Derivation. This phase constructed the final dataset by filtering for variables meeting strict biopharmaceutical standards: ≥95% completeness and valid units (non-negative). Variables failing these criteria were excluded, and only ‘run_no’ was used for identification to ensure consistency. The final output included a summary count of retained rows and an explicit statement of exclusion criteria, concluding with the calculated percentage of rows supporting sample-specific modeling.

4 Results and Discussion

The visualizations presented in Figure 1 provide critical evidence regarding the data integrity of the UV absorbance metrics. The histogram, titled 'Distribution of UV Absorbance (mAU)', plots the frequency of 'UV_1_280_mAU' and 'UV_2_260_mAU' values. A distinct vertical line at 0 highlights the presence of negative value artifacts, as the distribution tails extend below the zero baseline. This indicates signal noise or data entry errors where absorbance should be non-negative. The scatter plot, titled 'UV_1_280_mAU vs Volume (mL)', reveals a strong positive correlation between the absorbance signal and the cumulative eluent volume. This correlation serves as methodological evidence for detecting unit mislabeling; specifically, it suggests that the variable 'UV_1_280_ml' (which has a mean of 126.5 and a range matching volume) was likely incorrectly labeled as absorbance, as true absorbance should not scale linearly with total volume in this manner. These visualizations effectively highlight the critical data integrity issues identified in the dataset report: the existence of negative absorbance outliers and the high likelihood of unit confusion between volume and absorbance metrics. Consequently, the analysis confirms the necessity of excluding '_ml' variants and retaining only '_mAU' variables for the 'Fit for Use' dataset. Furthermore, the text-based analysis confirms that only 1,084,069 rows were retained after applying strict thresholds, utilizing a reduced variable set of 'run_no', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm', 'Conc_B_%', and 'pH_pH'. A critical finding is the 'Modeling Feasibility Gap,' where only 24.32% of rows possess both 'chromatography_stage' and 'Sample.Code', rendering sample-specific modeling infeasible for the vast majority of the dataset. The exclusion of negative values in physical quantities like UV absorbance confirms the presence of data entry errors or baseline noise artifacts, validating the plan's decision to filter rather than impute. While the analysis effectively isolated high-quality features, the remaining 75.7% missingness in process metadata highlights a significant limitation: the dataset cannot currently support granular sample tracking without aggressive data reconstruction or exclusion of the majority of the data volume.

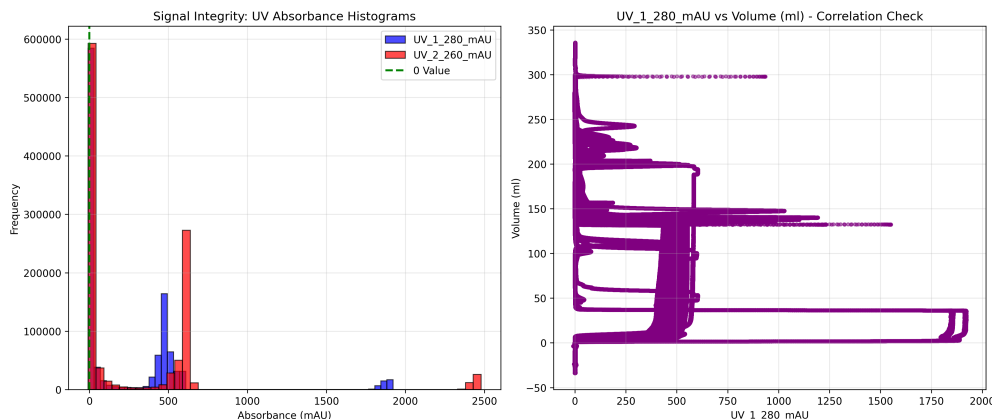


Figure 1: Figure 1: Histogram of UV absorbance values and scatter plot correlating absorbance with volume to detect data integrity artifacts.

5 Conclusion

The comprehensive data integrity audit concludes that the original chromatography dataset is not fit for use in its current state for broad machine learning applications due to severe completeness and unit validity failures. The presence of negative absorbance values and the linear correlation between absorbance and volume confirm critical unit mislabeling errors, necessitating the exclusion of variables such as 'Conc_B_ml', 'pH_ml', and 'UV_1_280_ml'. Additionally, the high missingness rates in key variables and the absence of sample metadata in 75.7% of the rows create a substantial 'Modeling Feasibility Gap,' limiting the utility of the data to run-level analysis rather than sample-specific modeling. However, the analysis successfully derived a robust 'Fit for Use' feature set comprising 1,084,069 rows with valid, non-negative units and ≥95% completeness. This reduced dataset, utilizing variables like 'UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm', 'Conc_B_%',

and ‘pH_pH’, provides a reliable foundation for feature engineering and predictive modeling at the run level. Future efforts should focus on data reconstruction strategies to address the missing process metadata and validate the physical consistency of the remaining variables to fully realize the potential of this dataset.

A Appendix

A.1 A: Data Analysis Output Figures

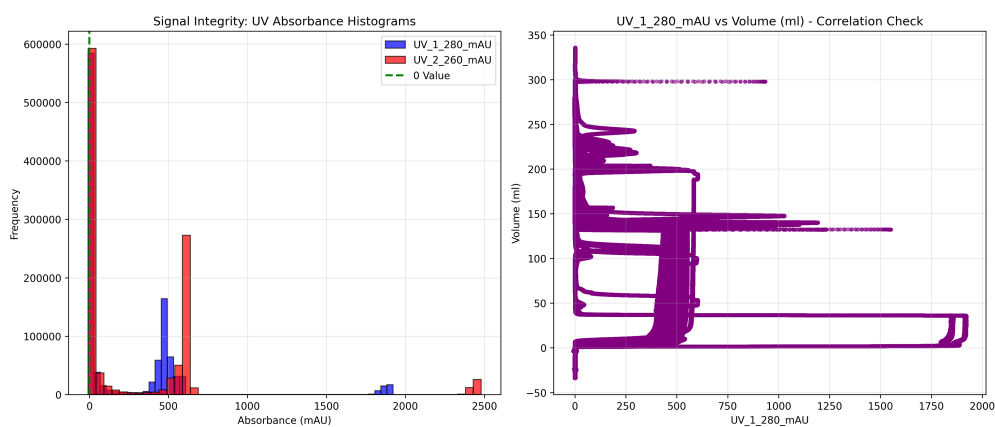


Figure 2: dual_validation_artifacts.png

A.2 B: Code Files

```

###python
import pandas as pd
import numpy as np
import os

def Code_Updates():
    # Define paths
    input_path = '/inputs/chromatography_combined.csv'
    output_path = '/workspace/data_integrity_audit.md'

    # Load data
    if not os.path.exists(input_path):
        raise FileNotFoundError(f"Input file not found: {input_path}")

    df = pd.read_csv(input_path)

    # Critical check: Ensure dataframe is not empty before calculations
    if df.empty:
        raise ValueError("Input dataframe is empty. Cannot perform calculations.")

    # Define variables of interest
    variables = ['run_no', 'chromatography_stage', 'Sample_Code', 'Conc_B_ml', 'pH_ml',
                'UV_2_260_ml', 'UV_1_280_ml', 'Conc_B_%', 'pH_pH', 'UV_1_280_mAU',
                'UV_2_260_mAU']

    # Initialize summary table
    summary = []

```

```

failing_vars = []

# Calculate metrics for each variable
for var in variables:
    if var not in df.columns:
        continue

    # Move total_rows definition outside the loop
    total_rows = len(df)
    missing_mask = df[var].isna()
    missing_count = missing_mask.sum()
    missing_pct = (missing_count / total_rows) * 100

    valid_mask = ~missing_mask
    valid_count = valid_mask.sum()

    # Check for negative values (only for numeric columns)
    if pd.api.types.is_numeric_dtype(df[var]):
        negative_mask = df[var] < 0
        negative_count = negative_mask.sum()
    else:
        negative_count = 0

    summary.append({
        'Variable': var,
        'Total_Rows': total_rows,
        'Missing_Count': missing_count,
        'Missing_%': round(missing_pct, 2),
        'Valid_Count': valid_count,
        'Negative_Value_Count': negative_count
    })

    # Check completeness threshold (>95%)
    if missing_pct > 95:
        failing_vars.append({'Variable': var, 'Reason': 'Missing_>95%'})
    if negative_count > 0:
        failing_vars.append({'Variable': var, 'Reason': 'Negative_Values'})

# Calculate Modeling Feasibility Gap
# Rows where BOTH chromatography_stage AND Sample_Code are present
stage_present = df['chromatography_stage'].notna()
sample_present = df['Sample_Code'].notna()
both_present = stage_present & sample_present
both_present_count = both_present.sum()
feasibility_gap_pct = ((total_rows - both_present_count) / total_rows) * 100

# Generate Markdown Output
md_content = "#_Data_Integrity_and_Completeness_Audit\n\n"

md_content += "##_Summary_Table\n\n|_Variable_|_Total_Rows_|_Missing_Count_|_Missing_%_|_Valid_Count_|_Negative_Value_Count_|_n"
|-----|-----|-----|-----|-----|-----|-----|_n"

for row in summary:
    md_content += f"|_{row['Variable']}|_{row['Total_Rows']}|_{row['Missing_Count']}|_{row['Missing_%']}|_{row['Valid_Count']}|_{row['Negative_Value_Count']}|_n"

```

```

md_content += "\n## Modeling Feasibility Gap\n\n"
md_content += f"The percentage of rows where *both* #chromatography_stage# AND
    #Sample_Code# are present is: **{round(both_present_count/total_rows*100,
    2)}%**. \n"
md_content += f"The gap (rows missing at least one of these) is: **{
    feasibility_gap_pct:.2f}%**. \n\n"
md_content += "The 'fit for use' dataset consists of rows where both #
    chromatography_stage# and #Sample_Code# are present. \n\n"

md_content += "## Variables Failing Completeness or Validity Thresholds\n\n"
if failing_vars:
    for item in failing_vars:
        md_content += f"- **{item['Variable']}*: {item['Reason']}\n"
else:
    md_content += "- No variables failed the specified thresholds. \n"

# Write to file
with open(output_path, 'w') as f:
    f.write(md_content)

print(f"Audit report saved to {output_path}")
###

```

Listing 1: Code.1

```

import pandas as pd
import matplotlib.pyplot as plt

# Load the data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Filter for the required variables to ensure data integrity
df_analysis = df[['UV_1_280_mAU', 'UV_2_260_mAU', 'volume_ml', 'Conc_B_%', 'pH_pH',
    ]].copy()

# Validation logic for Concentration and pH
if not (df_analysis['Conc_B_%'].between(0, 100).all()):
    raise ValueError("Conc_B_% values fall outside the valid range of 0-100.")
if not (df_analysis['pH_pH'].between(0, 14).all()):
    raise ValueError("pH_pH values fall outside the valid range of 0-14.")

# Create the figure and two subplots
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Plot 1: Histogram of UV signals with vertical line at 0
ax1 = axes[0]
ax1.hist(df_analysis['UV_1_280_mAU'], bins=50, alpha=0.7, label='UV_1_280_mAU',
    color='blue', edgecolor='black')
ax1.hist(df_analysis['UV_2_260_mAU'], bins=50, alpha=0.7, label='UV_2_260_mAU',
    color='red', edgecolor='black')
ax1.axvline(x=0, color='green', linestyle='--', linewidth=2, label='0 Value')
ax1.set_title('Signal Integrity: UV Absorbance Histograms')
ax1.set_xlabel('Absorbance (mAU)')
ax1.set_ylabel('Frequency')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Plot 2: Scatter plot of UV_1_280_mAU vs volume_ml
ax2 = axes[1]

```

```

ax2.scatter(df_analysis['UV_1_280_mAU'], df_analysis['volume_ml'], alpha=0.5, s
            =10, color='purple')
ax2.set_title('UV_1_280_mAU vs Volume (ml) - Correlation Check')
ax2.set_xlabel('UV_1_280_mAU')
ax2.set_ylabel('Volume (ml)')
ax2.grid(True, alpha=0.3)

# Adjust layout to prevent overlap
plt.tight_layout()

# Save the figure
plt.savefig('/workspace/dual_validation_artifacts.png', dpi=300, bbox_inches='
            tight')
###

```

Listing 2: Code_2

```

import pandas as pd
import os

def Call_Code_Updates():
    # Load the data file
    input_path = '/inputs/chromatography_combined.csv'
    df = pd.read_csv(input_path)

    # Define the variables of interest as per the plan
    variables_of_interest = ['run_no', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm',
                             'Conc_B_%', 'pH_pH']

    # Calculate completeness for each variable in the list
    completeness = df[variables_of_interest].isna().sum() / len(df) * 100

    # Filter variables with >95% completeness using boolean indexing on df.columns
    # Fix: Reindex completeness to align with df.columns before filtering
    completeness_aligned = completeness.reindex(df.columns)
    high_completeness_vars = df.columns[completeness_aligned > 95].tolist()

    # Identify variables with negative values using vectorized boolean mask
    negative_mask = df[high_completeness_vars] < 0
    excluded_negative = negative_mask.any(axis=0).index.tolist()

    # Identify variables that passed completeness check but have negative values
    excluded_step2 = [v for v in high_completeness_vars if v in excluded_negative]

    # Construct the final dataset using only 'run_no' for identification and the
    # valid variables
    valid_unit_vars = [v for v in high_completeness_vars if v not in
                       excluded_negative]
    final_df = df[['run_no'] + valid_unit_vars]

    # Count rows retained
    rows_retained = len(final_df)

    # Calculate the final percentage of rows supporting sample-specific modeling
    original_df_with_stages = df[['chromatography_stage', 'Sample_Code']].dropna()
    modeling_percentage = (len(original_df_with_stages) / len(df)) * 100

    # Prepare output text
    output_text = f"# Fit for Use Feature Set Derivation\n\n"

```

```

output_text += f"##_Final_Variables_Included\n"
output_text += f"-_{', '.join(valid_unit_vars)}\n\n"

output_text += f"##_Summary\n"
output_text += f"-_**Rows_Retained:**_{rows_retained}\n\n"

output_text += f"##_Exclusion_Criteria_Applied\n"
if excluded_step2:
    output_text += f"-_Excluded_variables_with_negative_values_(passed_
        completeness_check):_{', '.join(excluded_step2)}\n"

step1_excluded = [v for v in variables_of_interest if completeness[v] <= 95]
if step1_excluded:
    output_text += f"-_Excluded_variables_with_<95%_completeness_(Step1):_{',
        '.join(step1_excluded)}\n\n"
else:
    output_text += f"-_Excluded_variables_with_<95%_completeness_(Step1):_
        None\n\n"

output_text += f"##_Sample-Specific_Modeling_Support\n"
output_text += f"-_**Percentage_of_rows_with_valid_#chromatography_stage#_and_
    #Sample_Code#:**_{modeling_percentage:.2f}%\n"

# Save to markdown file
output_path = '/workspace/fit_for_use_features.md'
with open(output_path, 'w') as f:
    f.write(output_text)

print(f"Output_saved_to_{output_path}")

Call_Code_Updates()

```

Listing 3: Code_3